LAB-1

Md Ashraf

October 3, 2024

Introduction

In this lab, we explore different seismic data processing techniques using the `SeismicUnix` (SU) software. The tasks include synthetic data generation, visualization, and spectral analysis. This report is organized into sections covering the code used, the results, and discussions for each task.

1 Code

The following sections describe the steps taken in this lab along with the corresponding bash code used for data processing.

```
#!/bin/bash
# Generate synthetic data and visualize it
suplane | suxwigb &
# Visualization with titles and axis labels
suplane | suxwigb title="suplane test pattern" label1="time (s)" label2="trace number" &
# Save suplane output to a file and visualize it
suplane > junk.su
suxwigb < junk.su title="suplane test pattern" label1="time (s)" label2="trace number" &

# Generate suplane data, apply suspecfx, and visualize
suplane | suspecfx | suxwigb &
#make suplane data, write to a file
suplane > junk.su
#find the amplitude spectrum
suspecfx < junk.su > junk1.su
#View the output as wiggle traces
suxwigb < junk1.su &
ls
suswapbytes<sonar.su>sonar-test.su
suswapbytes<seismic.su>seismic-test.su
suswapbytes<radar.su>radar-test.su
suxwigb < sonar-test.su &
suxwigb < seismic-test.su &
suxwigb < radar-test.su &
#Image plot
suximage < sonar-test.su &
#Grey scale
suximage < sonar-test.su perc=99 &
#The perc=99 passes only those items of the 99th percentile and below in amplitude
suximage < sonar-test.su perc=99 legend=1
#Legend marking
suximage < sonar-test.su legend=1 & #This will show a grayscale bar.
suximage < sonar-test.su legend=1 perc=99 &
```

Listing 1: suplane plot with suxwigb

Results:

In this Bash script, I generate and visualize synthetic seismic data using Seismic Unix (SU) tools.

First, I create a basic visualization with suplane to get a quick look at the synthetic data. Then, I enhance this visualization by adding titles and axis labels, which makes it easier to interpret the results.

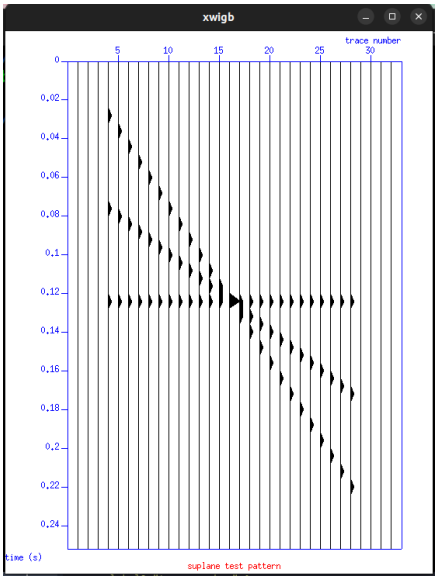
Next, I save the output from the suplane file in a file named junk.su. After that, I visualized this saved data with the same titles and labels.

To analyze the data further, I use suspecfx to calculate the amplitude spectrum, and I visualize this output as well. I also check the current directory to ensure that all my files are in order.

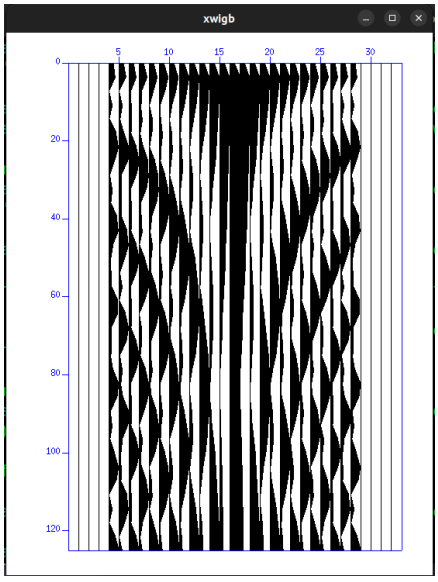
In addition to the synthetic data, I visualize several other datasets, including sonar.su, seismic.su, and radar.su, using suxwigb.

While working with seismic data on my system, I encountered an issue related to byte order. Since my machine runs on an x86 architecture, which is little endian, I had set my environment accordingly. However, I was dealing with a big endian SU file, and when I ran the surange command on this file, I received an error:

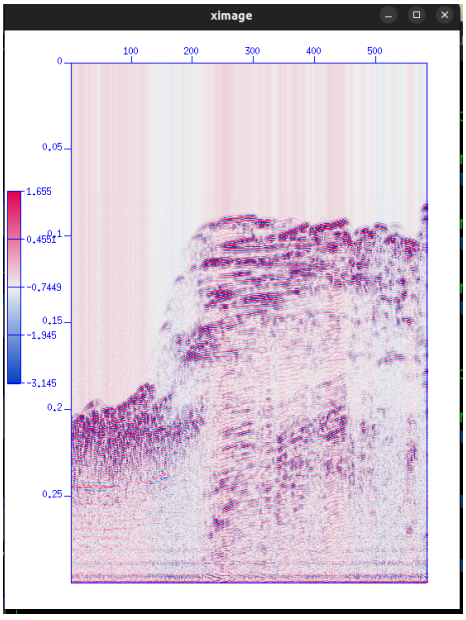
I soon realized it was due to a mismatch in the byte order between the SU file and my environment. To resolve this, I used the suswapbytes command, which conveniently swaps the byte order of SU data between big and little endian formats without needing any extra parameters. After swapping the bytes, surange worked perfectly on the new file.That's why I swapped the byte order of seismic.su, radar.su, and sonar.su, saving the output as seismic-test.su, radar-test.su, and sonar-test.su respectively. This allowed me to process all my data files without issues.



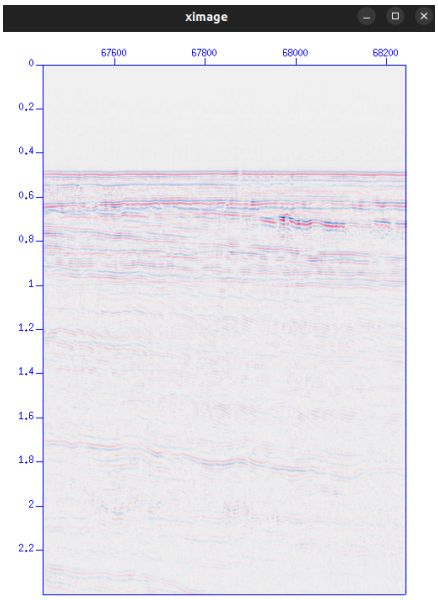
(a) Wiggle trace plot generated by **suplane**.



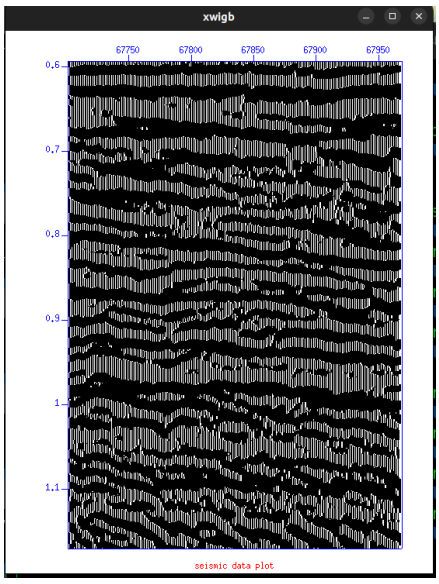
(b) Amplitude spectrum of the synthetic data.



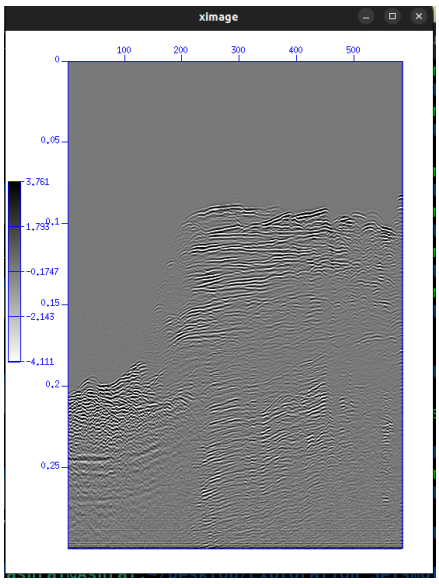
(c) Plot of sonar gain



(d) Plot of Seismic data.



(e) Wiggle Plot of seismic data.

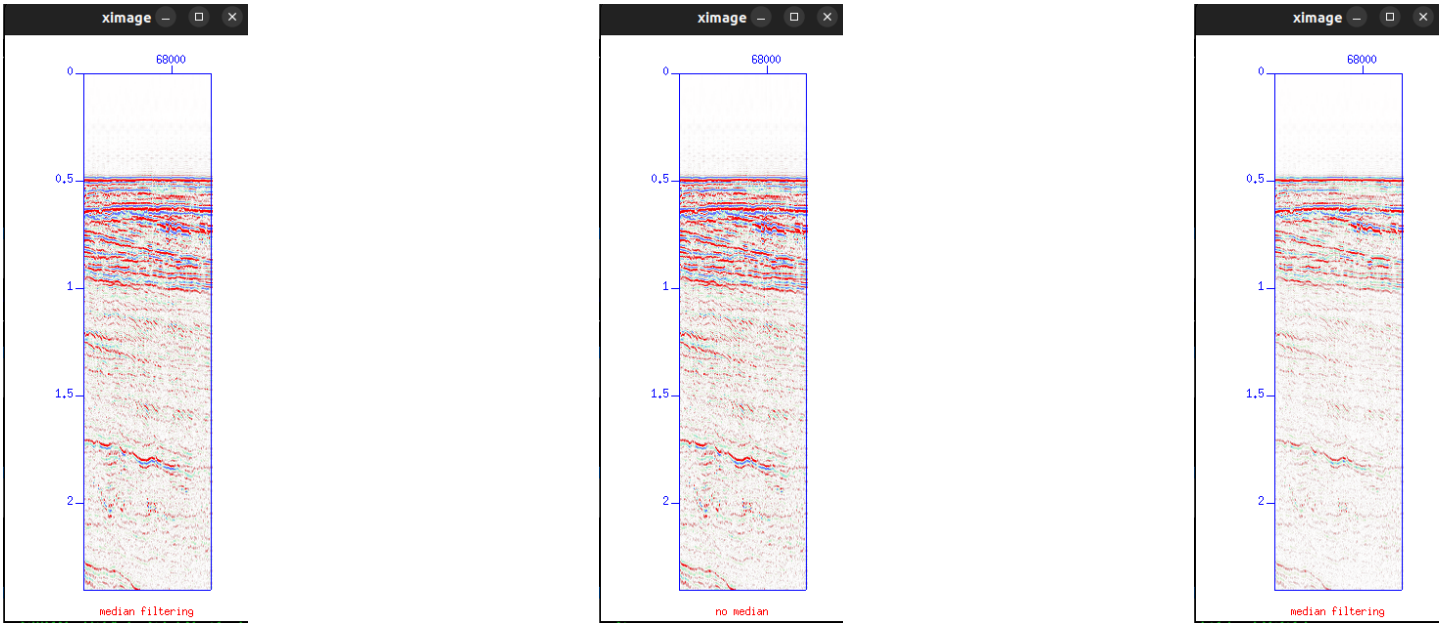


(f) Plot of sonar data on grey scale

```
#!/bin/bash

#Display balancing and display gaining
sunormalize norm=balmed < sonar-test.su | suximage legend=1
sunormalize norm=balmed < sonar-test.su | suximage legend=1 perc=99

sunormalize norm=balmed < sonar-test.su | sunormalize norm=rms | suximage legend=1

sunormalize norm=balmed < sonar-test.su | sunormalize norm=rms | suximage legend=1 perc=99

suximage < seismic-test.su wbox=250 hbox=600 cmap=hsv4 clip=3 title="no median" &

sunormalize norm=balmed < seismic-test.su | suximage wbox=250 hbox=600 cmap=hsv4 clip=3 title="
    ↪ median filtering" & #This result looks bizarre because the traces individually have
    ↪ different median values and consequently have different ranges of amplitudes.
sunormalize norm=balmed < seismic-test.su | sunormalize norm=rms | suximage wbox=250 hbox=600
    ↪ cmap=hsv4 clip=3 title="median filtering" &
```

Listing 2: suplane plot with suxwigb

2 Results:



(a) Seismic median filtering.

(b) Plot of seismic no median data.

(c) Plot of seismic normalized norm.

Figure 2: Results of seismic median filtering and normalization analyses.

3 Discussion

In this script, I applied display balancing and gain adjustments to seismic and sonar data. I used the ‘sunormalize‘ command with the ”balmed” normalization to balance the data based on the median, and I visualized the results using ‘suximage‘. I also filtered the data by using the ‘perc=99‘ option, which limited the display to the 99th percentile to remove extreme values.

For sonar data, I first normalized the data using the median normalization (‘norm=balmed‘), then displayed it. I applied root mean square (rms) normalization after median balancing to further smooth the data.

For seismic data, I used a similar approach, adjusting the display parameters such as window box size (‘wbox‘, ‘hbox‘) and color mapping (‘cmap=hsv4‘) to improve the visualization.